

LINUX ARM64 · TINYCONFIG · KERNEL 7.1-RC7

start_kernel() — 93 次子系统初始化逐帧分析

每一次调用前后，内核能做什么、不能做什么。基于 QEMU arm64 实际运行，从 init/main.c:1017 到 rest_init() 返回。

源文件 init/main.c · 1017-1229 行 函数体 213 行 (含空行 30 · 注释 17) 实际调用 ~93 个子系统 init

内存演进 memblock → buddy → slab

● 阶段边界 ● 纪元跃迁 (不可逆状态切换) ● 调用前缺失 / 不可用 ● 调用后获得 / 可用 ● 补充说明

PHASE 01 PID 0 安全护栏 · cgroup 预热

单 CPU · MMU 已开 · 中断未显式禁用 · 仅 memblock

set_task_stack_end_magic(&init_task)

kernel/fork.c:903

调用前

init_task 内核栈底无保护；栈溢出会默默破坏相邻内存

调用后

栈底写入 0x57AC6E9D；溢出可被 magic 校验检测

smp_setup_processor_id()

arch/arm64/kernel/setup.c

调用前

smp_processor_id() 返回值不确定

调用后

从 MPIDR_EL1 读取硬件 CPU ID 并缓存；CPU 自识别完成

debug_objects_early_init() 用静态数组建立早期对象生命周期追踪哈希表 (slab 前可用)

init_vmlinux_build_id() 将 .notes 段 Build ID 暴露给 vmlinux_build_id；crash dump 工具用它匹配符号表

cgroup_init_early()

kernel/cgroup/cgroup.c

调用前

init_task 没有 cgroup 归属；需要 early_init 的子系统未挂载

调用后

遍历 cgroup_subsys[]，调用每个子系统的 early_init()；init_task 被分配到 root cgroup

● 中断关闭 local_irq_disable() + early_boot_irqs_disabled = true — 直到 Phase 5 末尾才重新开启

PHASE 02 硬件拓扑 · 内存映射 · CPU 基础设施

中断已禁用 · 仅 memblock · 无 percpu · 无命令行

boot_cpu_init()

kernel/cpu.c:3160

调用前
cpu_online_mask / cpu_active_mask /
cpu_present_mask / cpu_possible_mask 全为空

调用后
CPU 0 同时设入四个 cpumask ; num_online_cpus() 返回 1 ; SMP 位图基线建立

page_address_init() 初始化 HIGHMEM 页地址哈希表 (arm64 无 HIGHMEM, 此为 no-op, 保留跨架构兼容)

setup_arch(&command_line) arch/arm64/kernel/setup.c:250

调用前
只有 head.S 最小页表 ; DTB 未解析 ; NUMA 拓扑未知 ; 无 KASLR ; 内存布局未知

调用后
DTB 完整遍历 ; memblock 填充所有物理内存段 ; 内核页表重建 (vmalloc/vmemmap 区域映射) ; KASLR 偏移应用 ; command_line 从 /chosen 提取 ; GIC 早期映射完成

start_kernel() 内最重的单次调用, 内部展开 50+ 子函数, 建立整个内存视图。

jump_label_init() kernel/jump_label.c

调用前
所有 static_key / static_branch 走默认慢路径 ;
__jump_table 未排序

调用后
跳转表排序 ; 可按需将 NOP ↔ JMP 热补丁 (用于 tracing、cgroup、lockdep 快速路径)

static_call_init() kernel/static_call_inline.c

调用前
静态调用站点指向默认 trampoline (间接调用)

调用后
站点可被 patch 为直接调用 ; 消除热路径间接跳转开销

early_security_init() 初始化 security_hook_heads ; 注册需要早期 init 的 LSM (lockdown、yama)

setup_per_cpu_areas() arch/arm64/kernel/smp.c

调用前
所有 CPU 共享同一静态 __per_cpu_load 段 ; per_cpu() 宏返回错误值

调用后
从 memblock 为每个 possible CPU 分配独立副本 ; 填充 __per_cpu_offset[] ; this_cpu_read() 正式可用

setup_nr_cpu_ids() 将 nr_cpu_ids 从 CONFIG_NR_CPUS 收缩到实际值 ; 节约 percpu 内存

smp_prepare_boot_cpu() · early_numa_node_init() · boot_cpu_hotplug_init()
arch hook 完成 boot CPU SMP 注册 ; 同步 NUMA node_online_map ; 将 boot CPU 置入 CPUHP_ONLINE 状态机

parse_early_param() · parse_args(...) init/main.c:1055

调用前
earlycon=、mem=、nokaslr 等参数未生效 ; earlycon 驱动未注册

调用后
early param 回调执行 ; earlycon 注册 (printk 输出到真实 UART) ; mem= 限制写入 memblock

random_init_early(command_line) drivers/char/random.c

调用前
ChaCha20 CRNG 未初始化 ; get_random_bytes() 返回零

调用后

用架构 RNG (RNDR_ELO) + 命令行哈希 + 时序 jitter 完成早期种子；基础随机性可用

PHASE 03 内存子系统诞生

中断禁用 · memblock 还活着 · slab 即将到来

setup_log_buf(0)

kernel/printk/printk.c

调用前
printk 用极小静态缓冲（默认 4KB）；早期日志随时溢出丢失

调用后
从 memblock 分配大缓冲（默认 256KB）；已有日志全部迁移；dmesg 不再丢失

vfs_caches_init_early()

fs/dcache.c

调用前
dcache 和 inode 哈希表不存在

调用后
从 memblock 按 RAM 大小分配两张哈希表；VFS 路径解析底层结构就位

sort_main_extable()

lib/extable.c

调用前
异常修复表 (__ex_table) 可能未排序；copy_from_user() 故障查表为线性搜索

调用后
按指令地址排序；search_extable() 二分查找 O(log n)；用户态访问故障修复安全

trap_init()

arch/arm64/kernel/traps.c

调用前
VBAR_EL1 指向 head.S 最小向量表；SError/Debug/SVE/MTE trap 未配置

调用后
完整 arm64 异常向量表设立；同步/异步异常全部有 handler；PAUTH/BTI/SVE trap 按 CPU feature 配置

mm_core_init()

mm/mm_init.c:2696

调用前
只有 **memblock**；无页分配器；无 slab；无 vmalloc；kmalloc() 不可调用

调用后
完整内存体系上线：memblock_free_all() 将所有空闲页移交 buddy；kmem_cache_init() 启动 SLUB；vmalloc_init() 完成；kmalloc() / vmalloc() 正式可用

最大的状态跃迁之一。memblock 被清空，全部物理页进入 buddy 管理，所有后续内核分配从这里开始。

mm_core_init_early() 为后续 VMA 操作准备早期缓存（在 mm_core_init 前做最小化预热）

maple_tree_init() 为 Maple Tree (Linux 6.1+ VMA 管理结构) 创建 kmem_cache；mmap()/munmap() 依赖此

poking_init() 建立代码热补丁隔离映射；kprobes / ftrace / static_call 的 text_poke_*() 之后才安全

ftrace_init()

kernel/trace/ftrace.c

调用前
函数入口 NOP sled 未建索引；动态函数追踪不可启用

调用后
所有 fentry 调用点建立索引；function_graph_tracer 可安装；kprobes 可注册

early_trace_init() 分配追踪环形缓冲；trace_printk() 不再丢失；tracepoint 框架部分激活

PHASE 04 调度器 · 并发基础设施 kmalloc 可用 · 中断仍禁用 · 单线程

sched_init() kernel/sched/core.c:8874

调用前
无 runqueue；无调度类；schedule() 必崩；current 只是 init_task 静态体

调用后
每 CPU 的 struct rq 初始化；CFS / RT / DL / Idle 调度类注册；root_task_group 创建；init_task 放入 runqueue；schedule() 可调用（但仍单线程）

radix_tree_init() 创建 radix_tree_node kmem_cache；页缓存 / IDR / XArray 的底层存储就绪

housekeeping_init() 处理 isolcpus；计算 housekeeping cpumask；unbound workqueue 将避开隔离 CPU

workqueue_init_early() kernel/workqueue.c:7956

调用前
system_wq、system_highpri_wq 等全局 workqueue 不存在；queue_work() 必崩

调用后
6 个内置 workqueue 创建；work item 可以入队（但还不能执行——需等 kthreadd 和 workqueue_init() 在 PID 1 里完成）

rcu_init() kernel/rcu/tree.c

调用前
rcu_read_lock() 是 no-op；call_rcu() 必崩；无锁数据结构无法安全跨 CPU 访问

调用后
Tree RCU 初始化；宽限期追踪激活；call_rcu() 可注册回调；整个内核的无锁读侧保护就位

kvfree_rcu_init() · trace_init() · context_tracking_init()

延迟 kvfree 批量回收就绪；全局 trace array 初始化（tracepoint 可 arm）；NO_HZ_FULL 用户/内核转换追踪激活

PHASE 05 中断 · 定时器 · 时间子系统 调度器就绪 · RCU 就绪 · 中断仍禁用 · 时钟未校准

early_irq_init() · init_IRQ() kernel/irq/irqdesc.c · arch/arm64/kernel/irq.c

调用前
irq_desc 哈希表未分配；GIC-v3 寄存器未映射；IPI 向量未分配；硬件无法路由中断

调用后
IRQ descriptor 哈希表就位；GIC-v3 distributor + redistributor 配置完成；per-CPU banked 寄存器初始化；IPI 向量 0-7 保留；硬件层面可投递中断

tick_init() · rcu_init_nohz() tick 基础设施注册；per-CPU tick_device 分配；RCU 宽限期追踪与 dynticks 集成

timers_init() · hrtimers_init() kernel/time/timer.c · hrtimer.c

调用前

add_timer()、hrtimer_start() 崩溃 ; msleep() /
usleep_range() 不可用

调用后

每 CPU 五级时间轮 (低精度) + 三个 hrtimer base
(MONOTONIC/REALTIME/BOOTTIME) 就绪 ; 所有睡眠 / 延
迟 API 可用

srcu_init() · softirq_init() · vdso_setup_data_pages()

可睡眠 RCU 完全可用 ; NET_RX/TX/TIMER/TASKLET 软中断向量注册 ; VDSO 数据页映射 (用户态免系统调用 gettimeofday 基础)

timekeeping_init()

kernel/time/timekeeping.c:1956

调用前

ktime_get() 返回垃圾 ; wall clock 未定义 ; jiffies 未对齐

调用后

clocksource 初始化 ; timekeeper 填充 ; ktime_get() 返
回单调纳秒 ; wall clock 设为 epoch ; 所有时间 API 的基础

time_init()

arch/arm64/kernel/time.c

调用前

arm64 架构定时器 (CNTPCT_ELO) 未注册为 clocksource /
clockevent

调用后

arm64 arch timer 注册为主 clocksource 和 per-CPU
clockevent ; 第一个 tick 即将可以触发

random_init()

drivers/char/random.c

调用前

CRNG 只有早期种子 ; 不够安全用于密钥生成

调用后

追加时序 jitter + hw RNG + 内存地址熵 ; CRNG 标记
"initialized" ; /dev/urandom 安全 ; 内核密钥生成可用

kfence_init() 分配 KFENCE guard-page 内存池 ; 后续 kmalloc 随机采样将分配置于 guard page 间, 检测 UAF/OOB

boot_init_stack_canary()

arch/arm64/include/asm/stackprotector.h

调用前

init_task 栈 canary 是编译期常量 ; -fstack-protector 保护
形同虚设

调用后

从 CRNG 取随机值写入 task_struct.stack_canary 和
thread-local canary ; 函数返回时会验证

perf_event_init() · profile_init() · call_function_init()

PMU 框架注册 (perf_event_open() 就绪) ; 内核 profiling 缓冲分配 ; per-CPU SMP call function 队列初始化
(smp_call_function_single() 就绪)

● 中断开启 local_irq_enable() — 硬件可随时打断内核 ; 调度器 tick 第一次触发 ; 系统进入真正的并发状态

PHASE 06 中断后服务 · 延迟校准

中断已开启 · slab 可用 · 调度器激活 · 单线程

kmem_cache_init_late()

mm/slub.c

调用前

SLUB 在早期模式：无 per-CPU slab，所有 kmalloc 走慢路径持全局锁

调用后

per-CPU slab cache 激活；NUMA-aware 分配列表建立；kmalloc 热路径完全展开

console_init()

drivers/tty/tty_io.c

调用前

只有 earlycon（直接写 UART MMIO）；无 TTY 层；无 /dev/console

调用后

从 __con_initcall 段注册完整控制台驱动（pl011、virtio-console 等）；TTY 核心初始化；printk 进入正式控制台路径

lockdep_init()

kernel/locking/lockdep.c

调用前

锁依赖追踪哈希表未分配；死锁检测无法工作

调用后

classhash_table 和 chainhash_table 分配；锁获取历史开始记录；循环死锁 / AA 死锁实时检测激活

locking_selftest() 在 lockdep 下运行 ~50 个锁反转场景；任何未被捕获的情况直接 BUG()

setup_per_cpu_pageset()

mm/page_alloc.c

调用前

buddy 每次分配都获取 zone->lock；高频单页分配存在严重锁竞争

调用后

每 CPU 的 per-page-cache (pcp) 初始化；order-0 页分配/释放绕过 zone 锁；内存分配热路径锁开销接近零

numa_policy_init() · acpi_early_init() · sched_clock_init()

创建 mempolicy kmem_cache；ACPI 总线扫描；校准每 CPU sched_clock() 基准（防止调度器计时跳变）

calibrate_delay()

init/calibrate.c

调用前

loops_per_jiffy = 0；udelay(N) / mdelay(N) 完全不准

调用后

在一个 jiffy 内计数循环次数，校准 loops_per_jiffy；BogoMIPS 打印到 dmesg；udeelay() 精确

arch_cpu_finalize_init()

arch/arm64/kernel/cpufeature.c

调用前

Spectre/Meltdown 等 CPU 漏洞缓解措施尚未应用到内核代码中

调用后

apply_alternatives_all() 执行：针对实际 CPU 型号将 NOP 替换为 SSBD/SB 等指令；BTI/MTE/PAN 按 feature bit 启用；内核 .text 段此后冻结

这是内核代码段的最后一次修改窗口。之后 mark_readonly() 将把内核文本段标为只读，不可再 patch。

PHASE 07 进程基础设施 · VFS · 命名空间

中断开启 · kmalloc 全速 · sched_clock 已校准 · 等待 fork() 完成

pid_idr_init() · anon_vma_init() · thread_stack_cache_init()

kernel/pid.c · mm/rmap.c · kernel/fork.c

调用前

调用后

PID 分配、匿名页反向映射、线程栈均无 kmem_cache

pid / anon_vma / anon_vma_chain / 线程栈的 kmem_cache 就绪；copy-on-write 逆映射可用

cred_init() · fork_init() · proc_caches_init()

kernel/cred.c · fork.c

调用前

task_struct / mm_struct / files / sighand / signal / cred 均无 kmem_cache；fork() 必崩

调用后

所有进程资源的 kmem_cache 创建；max_threads 按可用 RAM 计算；clone()/fork() 机制完整就绪

uts_ns_init() · time_ns_init()

创建 UTS 和 Time 命名空间 kmem_cache；uname()、sethostname()、时钟偏移命名空间隔离就绪

key_init() · security_init()

security/keys/key.c · security/security.c

调用前

内核密钥环不可用；LSM (SELinux/AppArmor/BPF LSM) 未加载

调用后

密钥类型表 + 用户密钥环 kmem_cache 就绪；所有 enabled LSM 的 init() 回调执行；安全钩子激活

net_ns_init()

net/core/net_namespace.c

调用前

init_net 命名空间不存在；sock_net() / dev_net() 返回 NULL

调用后

init_net 创建并注册；loopback 占位符添加；网络命名空间框架就绪；所有网络子系统可以初始化

vfs_caches_init()

fs/dcache.c:3401

调用前

只有 early 哈希表；dentry / inode / file / mount kmem_cache 均不存在；open()/read()/write() 不可用

调用后

dentry、inode、filp (struct file)、names_cache (路径名)、mnt_cache (mountpoint) 全部创建；mount namespace 初始化；rootfs 作为 "/" 伪根挂载；open() 系统调用基础设施完整

pagecache_init() · signals_init() · seq_file_init()

page cache per-inode 锁正确初始化；sigqueue kmem_cache 创建 (实时信号排队就绪)；seq_file kmem_cache (/proc 迭代接口就绪)

proc_root_init()

fs/proc/root.c

调用前

/proc 文件系统未挂载；task_struct 对外不可见

调用后

procs 注册并挂载；/proc/1/、/proc/cpuinfo、/proc/meminfo 等可访问；task_struct 通过 /proc/PID/ 暴露

nsfs_init() · pidfs_init()

挂载 namespace filesystem (setns()/unshare() 依赖) 和 pidfs (pidfd_open() 依赖)；容器运行时核心 API 就绪

cpuset_init() · mem_cgroup_init() · cgroup_init()

kernel/cgroup/cpuset.c · mm/memcontrol.c · kernel/cgroup/cgroup.c

调用前

只有 early cgroup ; /sys/fs/cgroup/ 不存在 ;
memory.limit_in_bytes 无效

调用后

cgroup v1/v2 文件系统挂载 ; /sys/fs/cgroup/ 层级建立 ;
per-memcg 内存计页和限制生效 ;
clone(CLONE_NEWCGROUP) 可用 ; **Kubernetes cgroup**
隔离基础完成

taskstats_init_early() · delayacct_init()

创建 taskstats 和 task_delay_info kmem_cache ; per-task CPU/IO/内存延迟统计从下一个 fork() 开始记录

PHASE 08 收尾 · 移交控制权

所有内核子系统就绪 · 等待 PID 1 诞生

acpi_subsystem_init() · arch_post_acpi_subsys_init() · kcsan_init()

ACPI 总线扫描完成、设备对象创建 ; arm64 IORT 处理完成 GIC/PCI 中断映射 ; KCSAN 数据竞争 watchpoint 激活

rest_init() ← start_kernel() 的最后一行有效调用，永不返回

创建 PID 1

user_mode_thread(kernel_init) →
PID 1 进入 runqueue 等待调度

创建 PID 2

kernel_thread(kthreadd) → PID 2 ,
所有后续内核线程的父进程

状态切换

system_state =
SYSTEM_SCHEDULING — RCU 调度
器正式启动

PID 0 退位

schedule_preempt_disabled() → 当
前上下文成为 idle 线程，永不再运行普
通代码

PID 1 接棒

kernel_init_freeable() → smp_init(),
do_basic_setup(),
kunit_run_all_tests()、挂载 rootfs

INIT 段释放

free_initmem() → __init 段 261 KB 归
还 buddy ; system_state =
SYSTEM_RUNNING